NTIZA

A Mobile Marketplace for Agricultural Equipment and Expert Services in Rwanda

Final Project Report: Front-End and Back-End Implementation

[Course name and code]

[Group number]

[Member 1 name and student ID]

[Member 2 name and student ID]

[Member 3 name and student ID]

[Member 4 name and student ID]

[Instructor name]

[Submission date]

1. Project Overview and Implemented Functionalities

Smallholder farmers in Rwanda often lose planting and harvesting windows because they cannot find a tractor, a pump or an agronomist at the right time and at a known price. NTIZA is a mobile application that brings equipment owners, agricultural experts, booking agents and farmers onto one marketplace. A farmer can discover machines near their district, book them for a chosen date and duration, pay with Mobile Money, follow the status of the request and take short training courses. The application was built in Flutter with Material 3 and uses Firebase Authentication and Cloud Firestore as its back end [1], [4], [6]. It was implemented from the approved Figma prototype of 34 screens, and it runs as an Android mobile application.

The application serves five kinds of users, and every role has its own navigation and dashboard. Table 1 summarises what each role can do. Every function listed in the table reads or writes real data in Firestore; no screen depends on hard-coded demonstration values.

Table 1. Functionality by role

Role	**Implemented functionality**
Visitor (guest)	Browse the equipment marketplace with search and filters, open equipment details, view experts and training courses.
Farmer	Register, verify the e-mail address, book equipment or an expert through a calendar, pay by MTN MoMo or Airtel Money (simulated), follow bookings, cancel a pending request, delete finished bookings from history, report a problem, and receive notifications.
Equipment owner	Create, edit, delete and change the status of equipment; accept, decline and complete booking requests; view earnings computed from completed bookings.
Expert	Publish weekly availability, fee and session length; accept or decline consultation requests; view earnings.
Booking agent	Maintain a private list of farmers (create, edit, delete), book equipment on their behalf and see the commission earned.
Administrator	Approve or reject partner applications, suspend, reactivate or delete users, resolve disputes and load demonstration data.
All users	Edit the profile and change settings (text size, notifications, preferred district) that are stored on the device.

[INSERT SCREENSHOT: welcome, marketplace, booking and owner dashboard side by side]

Figure 1. Main screens of the NTIZA application

1.1 Alignment with the Figma prototype

Each of the 34 prototype screens was rebuilt in Flutter using the colour, spacing and typography tokens sampled from the design (Inter typeface, 18 dp card radius, green primary colour). One deliberate deviation concerns accessibility: the green of the prototype (#16A34A) gives a contrast ratio of 3.3:1 with white text, which is below the 4.5:1 minimum of WCAG 2.1 success criterion 1.4.3 [8]. We therefore darkened the primary colour to #15803D (5.0:1) and adjusted the secondary text and status colours in the same way. Interactive elements keep a minimum touch target of 48 dp as recommended by Material Design [9]. All screens use scrollable layouts so that they do not overflow when the phone is rotated or when the screen is as small as 5 inches. [Replace with your own comparison screenshots: Figma vs app]

[INSERT SCREENSHOT: Figma design next to the implemented screen (2 examples)]

Figure 2. Design match between the Figma prototype and the Flutter implementation

2. GitHub Repository and Demo Video

The complete source code, the Firestore security rules, the index definitions and all documentation are available in the project repository. The commit history shows the contribution of each member.

- Repository: [https://github.com/murinziben/ntiza_mobile_app] (accessible to the instructor: [public / instructor added as collaborator])
- Demo video (10–15 minutes): [INSERT VIDEO LINK]
- Group contribution tracker: [INSERT TRACKER LINK]

3. Setup Instructions

The following steps build and run the application on a phone or an emulator. Web, desktop and Chrome builds are not supported because the project is a mobile application.

1. Install Flutter 3.24 or newer and Android Studio (or a physical Android phone with USB debugging). Run `flutter doctor` and resolve every issue it lists.
2. Clone the repository and run `flutter pub get` in the project folder.
3. In the Firebase console of project `ntiza-e9af2`, open Authentication, then Sign-in method, and enable **Email/Password** and **Google**. Add the SHA-1 and SHA-256 fingerprints of the debug key (`cd android && ./gradlew signingReport`) in the project settings and download `google-services.json` again into `android/app`.

4. Create the Cloud Firestore database, open the Rules tab, paste the contents of `firestore.rules` and publish. Deploy the indexes with `firebase deploy --only firestore:indexes`.
5. Start the application with `flutter run` and choose the emulator or the connected phone.
6. Create the first administrator: register a normal account, open the document `users/<uid>` in the Firestore console and set `role` to `admin` and `status` to `active`. Log in again and use More, Load demo data, to add sample equipment, experts and courses.
7. Build the release package with `flutter build apk --release`; the file is created in `build/app/outputs/flutter-apk/`.

4. Architecture and State Management

4.1 Clean Architecture

The code follows the layering proposed by Martin [3]. The **domain** layer is plain Dart without any Flutter or Firebase import. It contains the entities (for example `Equipment`, `Booking` and `AppUser`), the abstract repository interfaces and the use cases that hold validation and business rules, such as the pricing of a booking, the rule that equipment with open bookings cannot be deleted, and the rule that only finished bookings can be removed from a farmer's history. The **data** layer implements the interfaces with Firebase and SharedPreferences and contains the mappers that convert Firestore documents to entities. The **presentation** layer contains the screens, reusable widgets, Riverpod controllers and providers. Dependencies point inwards: presentation and data depend on the domain, never the other way round (Figure 3). Because the Firebase classes appear only in the data layer, the whole domain and controller logic can be tested with in-memory fakes, which is how most of our tests are written.

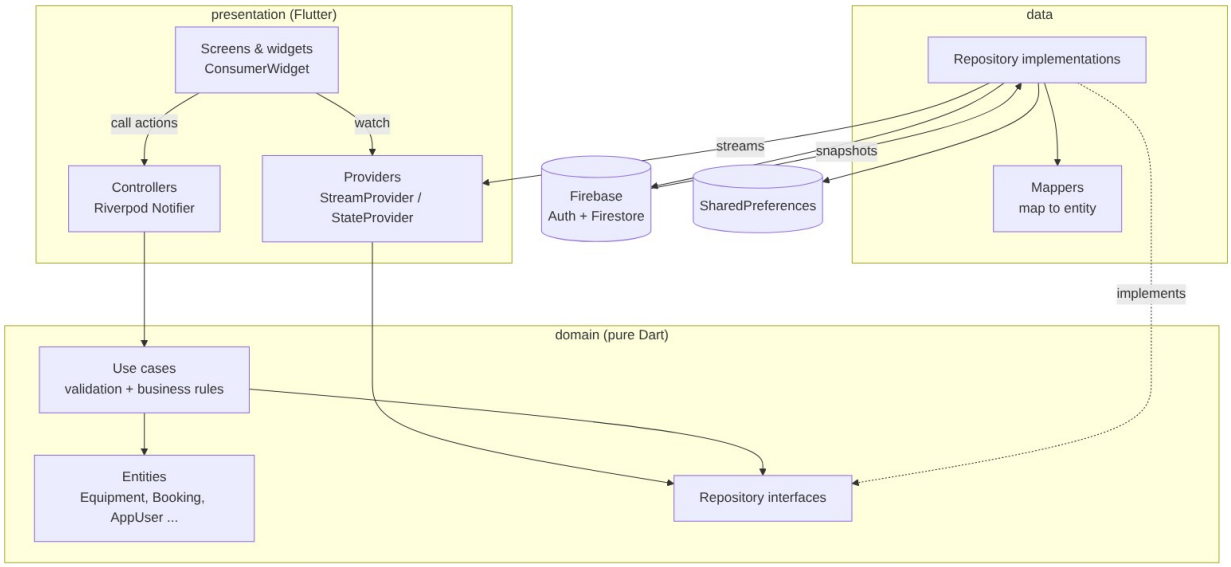


Figure 3. Layers of the application and the direction of dependencies

4.2 Why Riverpod

We chose Riverpod [2] for state management for four reasons. First, Firestore is naturally a stream: a `StreamProvider` turns a query into state that every screen can watch, so a change written by one user appears on another user's screen immediately and without a manual refresh. Second, providers are declared globally and are not tied to the widget tree, which allowed us to override the repositories with fakes in tests and to share the same state between widgets that are far apart; the bell badge on a dashboard and the notification list read the same provider. Third, providers can depend on other providers: the list of "my equipment" is derived from the list of all equipment and the current user, so it never goes out of date. Fourth, it removes the need for `setState`; the project contains no call to `setState`. Short-lived interface state, such as the selected filter chip or the "remember me" box, is stored in `StateProviders`, and actions that talk to the back end are `Notifier` controllers that return an `ActionResult` containing either a value or a friendly error message.

Figure 4 shows the flow of one action, booking a machine. The screen calls the controller, the controller calls a use case which validates the input and prices the job, the repository writes to Firestore in a transaction, and the new document flows back through a snapshot to the stream provider that feeds the My Bookings list.

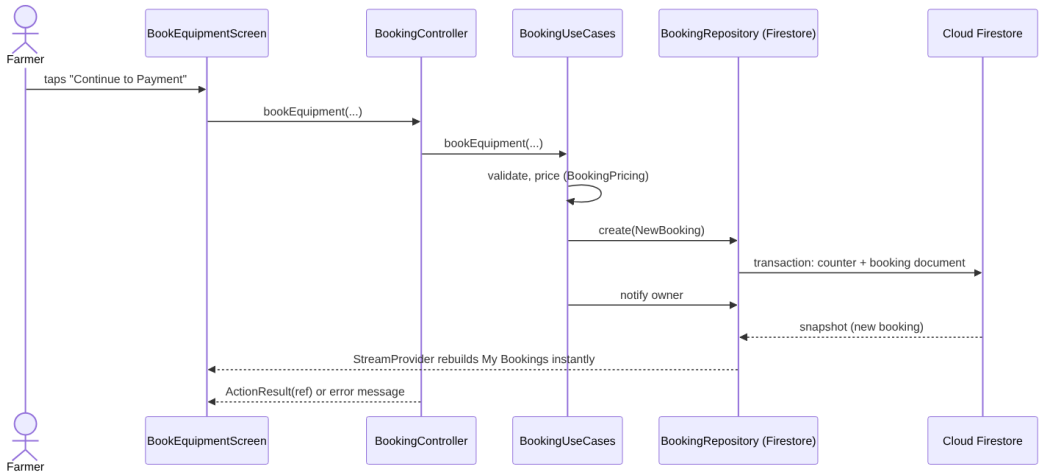


Figure 4. Sequence of the "book equipment" action through the layers

5. Database Design and Firebase Architecture

The database was designed before it was implemented. Figure 5 is the entity-relationship diagram of the implemented Cloud Firestore structure; each entity is a collection, each primary key is the document identifier, and each foreign key is a document identifier stored in a field. The field names in the diagram are exactly the names used by the mappers and by the security rules.

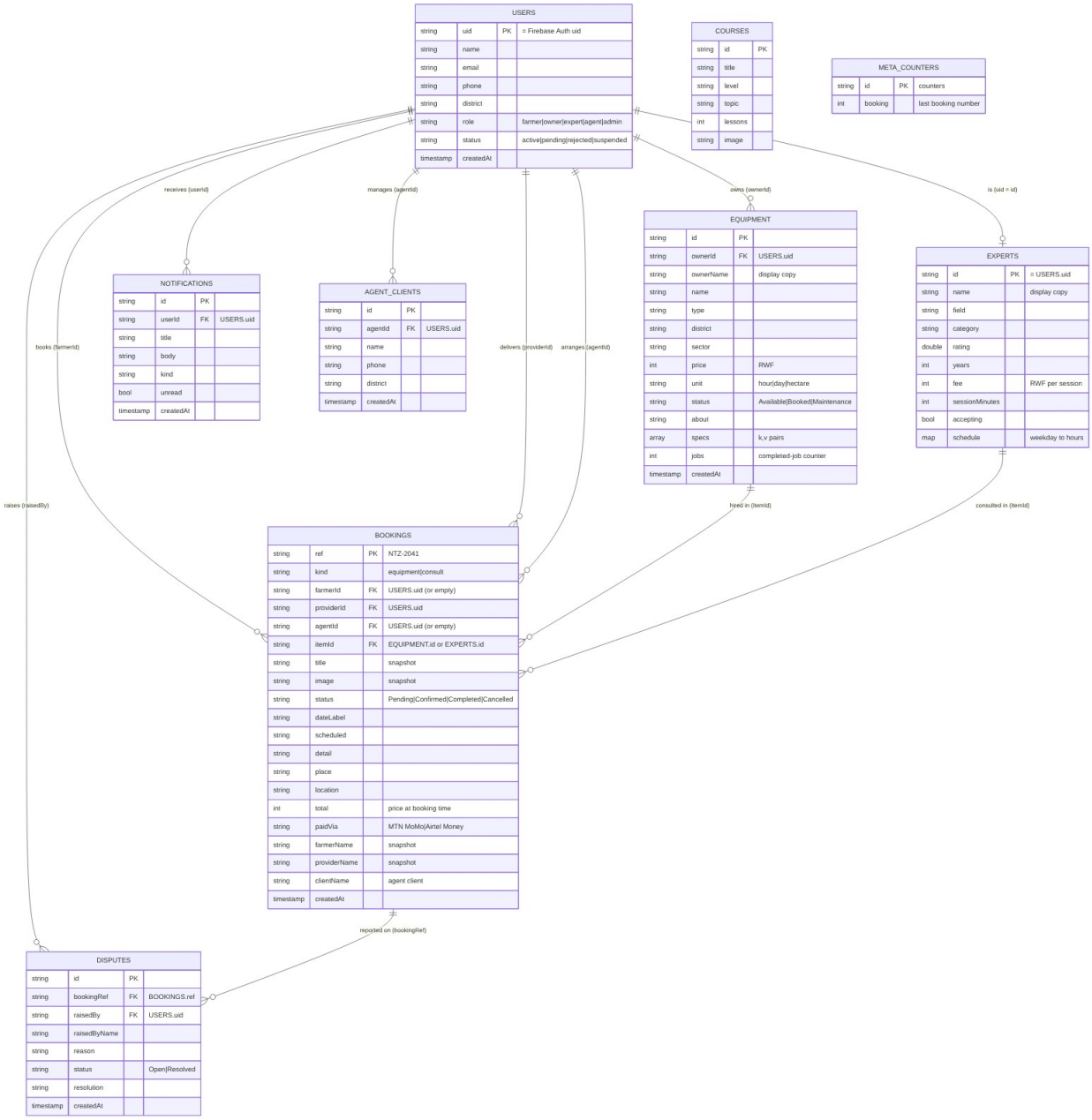


Figure 5. Entity-relationship diagram of the NTIZA Firestore database

Table 2. Collections, keys and relationships

Collection	Key	Foreign keys	Purpose
`users`	`uid` (Auth uid)	none	Profile, role and approval status
`equipment`	`id`	`ownerId` → users	Machines available for hire
`experts`	`id` (= users uid)	`id` → users	Fee, availability and rating of an expert

courses	id	none	Training courses
bookings	ref (NTZ-2041)	farmerId, providerId, agentId → users; itemId → equipment or experts	Hire or consultation request
notifications	id	userId → users	In-app alerts
disputes	id	bookingRef → bookings; raisedBy → users	Problems reported by users
agentClients	id	agentId → users	Farmers an agent books for
meta/counters	counters	none	Last booking number

5.1 Design decisions

Firestore has no joins, so relationships are expressed by identifiers and the application resolves them when needed. We avoided storing values that can be computed: the photo of a machine is derived from its type, and earnings, commission and chart values are calculated from the bookings. The only stored counter is equipment.jobs, which is incremented in the same operation that completes a booking. A booking keeps a snapshot of the title, the names of the two parties and the total price at the moment it was created. This is intentional, in the same way that an invoice keeps the price at the time of purchase: editing or deleting a machine later must not rewrite history. equipment.ownerName and experts.name are display copies that avoid one additional read for every row of a list.

Queries that filter on one participant and sort by creation time (bookings of a farmer, of a provider or of an agent, and the notifications of a user) need composite indexes. They are declared in firestore.indexes.json and deployed with the Firebase command line tool. The booking reference is produced by a Firestore transaction on meta/counters, which guarantees that two simultaneous bookings never receive the same number.

5.2 CRUD operations

Table 3. CRUD operations and the screens that perform them

Entity	Create	Read	Update	Delete
Equipment	Owner form	Marketplace, owner list	Edit form, status menu	Owner menu (blocked if open bookings)
Booking	Booking and	My Bookings,	Accept, decline,	Farmer history

	payment flow	detail, owner and admin lists	complete, cancel, pay	
Notification	Created by booking events	Notification list and badge	Mark read, mark all read	Swipe to delete
Agent client	Add farmer dialog	Farmers tab	Edit dialog	Delete with confirmation
Dispute	Report a problem	Admin list	Resolve with note	Admin delete
Profile / expert	Registration	Profile screen	Edit profile, availability	Admin delete user

[INSERT SCREENSHOT: Firestore console showing one created, updated and deleted document]

Figure 6. Firestore console after a create, update and delete performed from the app

6. Firebase Security Rules

Security rules are evaluated on the server for every read and write, so they protect the data even if somebody calls Firestore with a modified copy of the application [5]. Our rules start from "deny everything" and then allow only what each role needs. The most important rules are the following.

- **Profiles.** A user can read and edit only their own document, and only the fields `name`, `phone` and `district`. When the profile is created the role must be one of farmer, owner, expert or agent, and the status must be `active` for farmers and `pending` for partner roles. Nobody can create an administrator or change their own role or status. Only an administrator can list users, change a status or delete another user.
- **Equipment.** Reading is public so that visitors can browse. Creating requires an owner whose account is active, and the `ownerId` must be the caller. Updating and deleting are limited to the owner of the document or an administrator, and the counter `jobs` can only grow by one.
- **Bookings.** A booking can be read only by the farmer, the provider, the booking agent or an administrator. On creation the status must be `Pending` and the caller must be the farmer, or an active agent booking for a client. The provider may change only the `status` to confirmed, cancelled or completed. The farmer may set `paidVia` or cancel while the booking is still pending, and may delete only finished bookings.
- **Experts.** An approved expert can change only the fee, the session length, the accepting flag and the schedule of their own document.
- **Notifications, disputes and agent clients.** A user reads, updates and deletes only their own notifications; a dispute is visible to the person who raised it and to administrators, and only administrators resolve it; an agent sees only the clients they created.
- **Counter.** The booking counter can only increase by exactly one.

These rules map one-to-one to the ownership fields of the ERD (`farmerId`, `providerId`, `agentId`, `ownerId`, `userId`, `raisedBy`), which is why the ERD and the implementation had to use identical field names. Rules and role checks are complemented by route guards in the application: the router sends a signed-out user to the login screen, an unverified user to the verification screen, a pending or suspended account to a status screen, and keeps every role inside its own area.

A known weakness of the prototype is that any signed-in user can create a notification for another user, because the notification is written by the client. In production this write should be moved to a Cloud Function (see Section 10).

7. Authentication

The application supports two authentication methods. The first is e-mail and password. Registration collects the name, e-mail, phone number, district and a password that must have at least eight characters with a letter and a number; the form shows an error under each field as soon as the user leaves it. After registration Firebase sends a verification e-mail, and the user must confirm it before the application opens, which prevents accounts with e-mail addresses that belong to somebody else. A "Forgot password?" screen sends a reset link. The second method is Google sign-in through Firebase; a new Google user is asked to choose a role and complete the profile before entering. The Firebase session is persisted by the SDK, so the user remains signed in after the application restarts. Logging out clears the session, and because all providers depend on the current user, the screens are cleared of the previous user's data. Error messages are translated from Firebase error codes into sentences the user can act on, for example "Incorrect e-mail or password" or "No internet connection". [Add screenshots of register validation, verification screen and Google sign-in]

[INSERT SCREENSHOT: registration with validation errors; verify-email screen; Google account chooser]

Figure 7. Authentication flows

8. User Preferences with SharedPreferences

Five preferences are stored on the device with the `shared_preferences` package [11] and are restored when the application starts: the text size (small, normal or large), whether notifications are shown, the preferred district that pre-selects the marketplace filter, the e-mail address remembered by "Remember me", and whether the welcome screen has already been seen. The `SettingsController` loads the saved values synchronously at start-up, because `main()` waits for the preferences before the first frame, and it saves every change immediately. The text size is applied to the entire application through the `MediaQuery` text scaler in the root widget.

9. Testing Performed and Results

The project contains 107 automated tests (96 unit and controller tests and 11 widget tests) in 9 files, written with `flutter_test` [10]. Because the domain and controller layers do not depend on Firebase, the repositories are replaced by in-memory fakes, so the tests run in seconds on a development machine. Table 4 lists what is covered.

Table 4. Automated tests

Test file	Tests	What it verifies
`core/formatters_test.dart`	5	
`core/router_redirect_test.dart`	10	
`data/mappers_test.dart`	7	
`data/settings_repository_test.dart`	3	
`domain/pricing_test.dart`	10	
`domain/usecases_test.dart`	27	
`domain/validators_test.dart`	13	
`presentation/controllers_test.dart`	21	
`presentation/widgets_test.dart`	11	

Test results: [INSERT the output of `flutter test` — e.g. "All tests passed! (NN tests)"]. Code coverage measured with `flutter test --coverage` is [INSERT PERCENT]%. [Replace with your numbers]

[INSERT SCREENSHOT: terminal showing "All tests passed!"]

Figure 8. Result of `flutter test`

[INSERT SCREENSHOT: coverage summary (lcov or genhtml output)]

Figure 9. Test coverage report

In addition to the automated tests, we tested manually on [device model and Android version] and on [emulator name]. We checked every screen in portrait and landscape, on a small (5-inch) and a large (6.7-inch) screen, with the three text sizes, and we repeated the complete booking

flow for each role. `dart format` and `flutter analyze` were run before submission; the analyzer reports [0 issues — INSERT SCREENSHOT].

[INSERT SCREENSHOT: output of flutter analyze: No issues found!]

Figure 10. Static analysis result

10. Known Limitations and Future Work

10.1 Known limitations

- **Mobile Money is simulated.** The payment screen waits two seconds and records the provider on the booking. A real charge requires merchant credentials from MTN or Airtel and a server component that receives the payment callback.
- **No image upload.** The photo of a machine is chosen from its type; Firebase Storage is not integrated.
- **No push notifications.** Notifications are stored in Firestore and shown inside the application.
- **Client-side counters and notifications.** The booking number and the notifications are written by the client. The rules limit the damage but a Cloud Function is the correct place for both.
- **Single language and light theme.** The interface is in English only and has no dark theme.
- **Ratings and reviews** are not yet collected from completed jobs.
- [Add any limitation you observed during your own testing]

10.2 Future work

- Integrate MTN MoMo and Airtel Money through a Cloud Function and webhook, and add payouts for owners and experts.
- Add photo upload with Firebase Storage and image compression for low-bandwidth connections.
- Add push notifications with Firebase Cloud Messaging and offline persistence of bookings.
- Translate the interface to Kinyarwanda and French and add a dark theme.
- Add ratings, maps with distance to the equipment, and calendar availability for each machine.
- Add integration tests that run the main flows on a device.

11. Methodology, Team Contributions and Use of AI

11.1 Methodology

The team followed an iterative process: prototype in Figma, entity-relationship design, domain layer, data layer, screens, then tests and documentation. Work was divided by feature and each feature was developed on its own Git branch and merged through a pull request reviewed by

another member. [Describe your sprints, meetings and how you divided the work in your own words.]

11.2 Contribution of each member

Table 5. Contributions (must match `git shortlog -sn` and the Group Contribution Tracker)

Member	Main responsibilities	Commits	Pull requests reviewed
[Member 1]	[e.g. authentication, settings]	[n]	[n]
[Member 2]	[e.g. farmer flows]	[n]	[n]
[Member 3]	[e.g. owner and expert]	[n]	[n]
[Member 4]	[e.g. admin, agent, tests]	[n]	[n]

11.3 Disclosure of AI assistance

The course requires AI assistance to be disclosed. During this project the team used the AI assistant Claude (Anthropic) [12] as a development aid. [State exactly what it was used for, for example: generating the first version of the Firebase repositories, drafting unit tests, reviewing code for analyzer problems, and drafting this report.] The team estimates that AI-generated or AI-assisted text and code make up approximately [INSERT YOUR HONEST ESTIMATE] % of the submission. All generated code was read, run, tested and, where necessary, modified by the members, who can explain every part of it. [Edit this paragraph so that it is true for your group; the figure must be your own honest estimate.]

References

[1] Flutter Team, "Flutter documentation," Google. [Online]. Available: https://docs.flutter.dev. [Accessed: Nov. 2026].

[2] R. Boelens, "Riverpod: documentation," 2024. [Online]. Available: https://riverpod.dev. [Accessed: Nov. 2026].

[3] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River, NJ, USA: Prentice Hall, 2017.

[4] Google, "Cloud Firestore documentation," Firebase. [Online]. Available: https://firebase.google.com/docs/firestore. [Accessed: Nov. 2026].

[5] Google, "Firebase Security Rules documentation," Firebase. [Online]. Available: https://firebase.google.com/docs/rules. [Accessed: Nov. 2026].

[6] Google, "Firebase Authentication documentation," Firebase. [Online]. Available: https://firebase.google.com/docs/auth. [Accessed: Nov. 2026].

[7] Flutter Team, "go_router package," pub.dev. [Online]. Available: https://pub.dev/packages/go_router. [Accessed: Nov. 2026].

[8] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," Success Criterion 1.4.3 Contrast (Minimum), 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Nov. 2026].

[9] Google, "Material Design 3: Accessibility and touch targets." [Online]. Available: https://m3.material.io. [Accessed: Nov. 2026].

[10] Flutter Team, "Testing Flutter apps." [Online]. Available: https://docs.flutter.dev/testing/overview. [Accessed: Nov. 2026].

[11] Flutter Team, "shared_preferences package," pub.dev. [Online]. Available: https://pub.dev/packages/shared_preferences. [Accessed: Nov. 2026].

[12] Anthropic, "Claude," AI assistant. [Online]. Available: https://www.anthropic.com/claude. [Accessed: Nov. 2026].